Finance App

Architecture, Best Practices & Docker Guide

Laravel 13 • Vue 3 • TypeScript • PostgreSQL
Generated April 2026

Table of Contents

1. Project Overview

1.1 Tech Stack

1.2 Architecture Diagram

1.3 Directory Structure

2. Backend Improvements

2.1 BelongsToUser Global Scope

2.2 SavingsGoalStatus Enum

2.3 Controller Cleanup

2.4 Form Request Authorization

2.5 Database Indexes

2.6 Rate Limiting & Route Security

2.7 Exception Handling

3. Frontend Improvements

3.1 HTTP Error Interceptor

3.2 Router Guard Race Condition

3.3 Route Meta Types

3.4 Error Boundary

3.5 Typed Error Handling

3.6 Component Fixes

3.7 Configurable Locale

4. Docker Setup

4.1 Architecture

4.2 Dockerfile (Multi-Stage)

4.3 Nginx Configuration

4.4 Docker Compose (Production)

4.5 Docker Compose (Development)

4.6 Deployment Guide

5. File Reference

1. Project Overview

The Finance App is a personal finance management tool that helps users track income, expenses, budgets, and savings goals. It is built as a **single-page application (SPA)** with a decoupled backend API.

1.1 Tech Stack

Layer	Technology	Version
Backend Framework	Laravel	13.x
Language (Backend)	PHP	8.3+
Frontend Framework	Vue 3 (Composition API)	3.5
Language (Frontend)	TypeScript	5.x
State Management	Pinia	3.x
Routing (Frontend)	Vue Router	5.x
HTTP Client	Axios	1.x
CSS	Tailwind CSS	4.x
Build Tool	Vite	8.x
Database	PostgreSQL	17
Authentication	Laravel Sanctum (SPA mode)	4.x
Session / Cache / Queue	Database driver	—

1.2 Architecture Diagram

```
Browser (Vue 3 SPA)
    |
    |  HTTP requests (same origin)
    v
Nginx (:80 / :443)
    |
    |--- /build/*          --> Static files (Vite-built JS/CSS, immutable cache)
    |--- /api/*            --> FastCGI to PHP-FPM (:9000)
    |--- /sanctum/*        --> FastCGI to PHP-FPM (:9000)
    |--- /*                --> try_files, then fallback to index.php (SPA)
```

```
    |
    v
PHP-FPM (Laravel 13)
    |
    |--- Sanctum Auth (session cookies + CSRF)
    |--- API Controllers --> Form Requests --> Policies
    |--- Eloquent Models (global user scoping)
    |
    v
PostgreSQL (:5432)
    |
    |--- Application tables (users, categories, incomes, expenses, budgets, savings)
    |--- Framework tables (sessions, cache, jobs, failed_jobs)
```

1.3 Directory Structure

```
app/
├── Enums/                    # PHP enums (SavingsGoalStatus)
├── Http/
│   ├── Controllers/Api/      # 7 API controllers
│   ├── Requests/             # Form validation + authorization
│   └── Resources/            # API response formatting
├── Models/
│   └── Concerns/             # BelongsToUser trait (global scope)
├── Policies/                 # Ownership authorization
└── Services/                 # Business logic (DashboardService)

resources/js/
├── components/               # Reusable Vue components
│   ├── common/               # AppHeader, Sidebar, Toast, MonthYearPicker
│   ├── dashboard/            # SummaryCards, BudgetOverview, etc.
│   └── forms/                # CategoryForm, ExpenseForm, BudgetForm, etc.
├── composables/              # useNotification store
├── layouts/                  # AppLayout, AuthLayout
├── pages/                    # Route-level page components
├── router/                   # Vue Router config + guards
├── services/                 # Axios HTTP service layer
├── stores/                   # Pinia state management
├── types/                    # TypeScript interfaces
└── utils/                    # Formatters, constants, API error helpers

docker/
├── nginx.conf                # Nginx server config
└── entrypoint.sh             # Container boot script
```

2. Backend Improvements

2.1 BelongsToUser Global Scope

File: `app/Models/Concerns/BelongsToUser.php`

Problem: Every controller query manually filtered with `->where('user_id', $request->user()->id)`. This is repetitive and, more critically, if a developer forgets the filter in a new query, it creates a data leak where users can see each other's records.

Solution: A reusable trait that adds a global scope to automatically filter all queries by `Auth::id()`. It also auto-sets `user_id` on newly created models.

```
trait BelongsToUser
{
    protected static function bootBelongsToUser(): void
    {
        static::addGlobalScope('user', function (Builder $query) {
            if (Auth::check()) {
                $query->where($query->getModel()->getTable() . '.user_id', Auth::id());
            }
        });

        static::creating(function (Model $model) {
            if (Auth::check() && ! $model->user_id) {
                $model->user_id = Auth::id();
            }
        });
    }
}
```

Applied to: Category, Income, Expense, Budget, SavingsGoal

Impact: Controllers no longer need to manually pass `user_id` during creation or filter by it during queries. For example, creating an expense simplifies from:

```
// Before
$expense = Expense::create([
    'user_id' => $request->user()->id,
    ...$request->validated(),
]);

// After
$expense = Expense::create($request->validated());
```

2.2 SavingsGoalStatus Enum

File: `app/Enums/SavingsGoalStatus.php`

Problem: The `status` field on SavingsGoal was a bare string with no model-level type safety. Invalid values like `"pending"` could be written without error.

Solution: A PHP 8.3 backed enum, cast on the model:

```
enum SavingsGoalStatus: string
{
    case Active = 'active';
    case Completed = 'completed';
    case Cancelled = 'cancelled';
}

// In SavingsGoal model:
protected $casts = [
    'status' => SavingsGoalStatus::class,
];
```

Now assigning an invalid status throws a `ValueError` instead of silently corrupting data.

2.3 Controller Cleanup

Removed unnecessary `fresh()` calls

Affected: BudgetController, ExpenseController, CategoryController, SavingsGoalController, IncomeController, SavingsContributionController

After calling `$model->update()`, the model instance is already synced. Calling `$model->fresh()` fires an extra `SELECT` query for no benefit.

```
// Before (extra query)
return new BudgetResource($budget->fresh()->load('category:id,name'));

// After (no extra query)
$budget->load('category:id,name');
return new BudgetResource($budget);
```

Extracted duplicate expense aggregation

Both `BudgetController` and `DashboardService` had identical queries to sum expenses by category. Extracted to a shared static method:

```
// BudgetController
public static function getActualByCategory(int $month, int $year): Collection
{
    return Expense::whereMonth('date', $month)
```

```
        ->whereYear('date', $year)
        ->selectRaw('category_id, SUM(amount) as total')
        ->groupBy('category_id')
        ->pluck('total', 'category_id');
}

// DashboardService now calls:
$actualByCategory = BudgetController::getActualByCategory($month, $year);
```

2.4 Form Request Authorization

Problem: All Form Requests returned `authorize() → true` while controllers separately called `$this->authorize()`. This split responsibility and meant validation ran before authorization—wasting resources on invalid requests.

Solution: Update requests now handle authorization via policies directly:

```
// UpdateExpenseRequest
public function authorize(): bool
{
    return $this->user()->can('update', $this->route('expense'));
}

// Controller no longer needs:
// $this->authorize('update', $expense);  ← removed
```

Applied to: UpdateCategoryRequest, UpdateExpenseRequest, UpdateIncomeRequest, UpdateBudgetRequest, UpdateSavingsGoalRequest

2.5 Database Indexes

File: `database/migrations/2024_01_01_000007_add_missing_indexes.php`

Added composite indexes for queries that filter by multiple columns:

Table	Index Columns	Query Pattern
savings_goals	(user_id, status)	Dashboard: active goals per user
savings_contributions	(savings_goal_id, contribution_date)	Contributions sorted by date
budgets	(user_id, month, year)	Budgets filtered by period
incomes	(user_id, date)	Income list sorted by date

The expenses table already had indexes on `(user_id, date)` and `(user_id, category_id, date)` from the original migration.

2.6 Rate Limiting & Route Security

File: `routes/api.php`

- **Auth endpoints** (login, register): `throttle:6,1` — 6 requests per minute. Prevents brute-force attacks.
- **Authenticated routes**: `throttle:60,1` — 60 requests per minute per user.
- **Scoped nested resources**: `savings-goals.contributions` now uses `->scoped()` which ensures a contribution's parent savings goal matches the route parameter, preventing access to contributions via a different goal's URL.

2.7 Exception Handling

File: `bootstrap/app.php`

Added a custom renderer for `NotFoundHttpException` on API routes. Previously, a 404 on `/api/expenses/999` would return an HTML error page. Now it returns proper JSON:

```
{ "message": "Resource not found." }
```

3. Frontend Improvements

3.1 HTTP Error Interceptor

File: `resources/js/services/http.ts`

Problem: The Axios instance had no error handling. A 401 response (expired session) would silently fail, leaving the user on a broken page with no feedback.

Solution: Added a response interceptor and a 30-second request timeout:

Status	Action	Reason
401 Unauthorized	Redirect to login page	Session expired or user logged out elsewhere
419 CSRF Mismatch	Reload the page	CSRF token expired; reload fetches a fresh token
Timeout (30s)	Reject promise	Prevents indefinite loading states

3.2 Router Guard Race Condition

File: `resources/js/router/guards.ts`

Problem: If the user navigated rapidly (e.g., clicked a link during initial page load), multiple concurrent `fetchUser()` calls could fire. This wasted requests and could cause the auth check to evaluate before the first fetch completed.

Solution: Deduplicates the fetch with a module-level promise reference:

```
let fetchingUser: Promise<void> | null = null;

router.beforeEach(async (to) => {
    if (!authStore.checked) {
        if (!fetchingUser) {
            fetchingUser = authStore.fetchUser();
        }
        await fetchingUser;  // All concurrent navigations wait on the same promise
    }
    // ... auth checks
});
```

3.3 Route Meta Types

File: `resources/js/types/router.d.ts`

Augments Vue Router's `RouteMeta` interface so that `meta.requiresAuth`, `meta.guest`, and `meta.layout` are type-checked. A typo like `meta.requireAuth` is now caught at compile time.

```
declare module 'vue-router' {
    interface RouteMeta {
        layout?: 'auth' | 'app';
        requiresAuth?: boolean;
        guest?: boolean;
    }
}
```

3.4 Error Boundary

File: `resources/js/App.vue`

Problem: If a lazy-loaded page component threw an error during render, the entire app would crash with a white screen and no recovery path.

Solution: Added `onErrorCaptured` at the root component. When an unhandled error occurs, a fallback UI is shown with a "Reload page" button instead of a blank screen.

3.5 Typed Error Handling

File: `resources/js/utils/api-error.ts`

Problem: Catch blocks used `err: any`, defeating TypeScript. ESLint had `no-explicit-any` disabled globally.

Solution: Created typed error utilities and changed the ESLint rule to `warn`:

```
// Type-safe error handling
import { getErrorMessage } from '@/utils/api-error';

try {
    await store.remove(id);
} catch (err: unknown) {
    notify.error('Error', getErrorMessage(err, 'Failed to delete.'));
}
```

Also removed the `[key: string]: unknown` index signature from the `User` type, which previously allowed any arbitrary property without a type error.

3.6 Component Fixes

Component	Fix
ContributionForm	Form no longer resets optimistically before knowing if the API call succeeded. Exposes `resetForm()` via `defineExpose`, called by the parent after a successful response.

MonthYearPicker	Added `aria-label="Previous month"` and `aria-label="Next month"` to icon-only navigation buttons for screen reader accessibility.
BudgetForm + MonthYearPicker	Extracted the duplicated months array to a shared constant in `utils/constants.ts`.
Expense types	Added a `PaymentMethod` union type (`'cash' | 'card' | 'bank_transfer' | 'mobile_money' | 'other'`) instead of bare `string`.

3.7 Configurable Locale

File: `resources/js/utils/formatters.ts`

Currency and date formatting previously hardcoded `'en-ZM'` and `'ZMW'`. Now reads from environment variables with fallback defaults:

```
const LOCALE = import.meta.env.VITE_LOCALE ?? 'en-ZM';
const CURRENCY = import.meta.env.VITE_CURRENCY ?? 'ZMW';
```

Set in `.env`: `VITE_LOCALE=en-ZM` and `VITE_CURRENCY=ZMW`.

4. Docker Setup

4.1 Architecture

```
                  docker-compose.yml
                  ====================


     ┌──────────┐      ┌──────────┐      ┌────────────────┐
     │  nginx   │──────►│   app    │──────►│      db        │
     │  :80     │      │ PHP-FPM  │      │ PostgreSQL     │
     │          │      │  :9000   │      │   :5432        │
     └──────────┘      └──────────┘      └────────────────┘
          │                 │
          │                 │            ┌────────────────┐
          │                 └────────────►│ queue-worker   │
          │                              │ artisan work   │
          ▼                              └────────────────┘
    app-public (shared volume)

             docker-compose.override.yml (dev only)
             ======================================


     ┌──────────┐
     │ node-dev │  Vite HMR server
     │  :5173   │  (bind-mounted source)
     └──────────┘
```

Service	Image	Port	Role
app	Custom (Dockerfile)	9000 (internal)	Laravel PHP-FPM, runs migrations on boot
nginx	nginx:1.27-alpine	80 (prod) / 8080 (dev)	Reverse proxy, static files, SPA fallback
db	postgres:17-alpine	internal (prod) / 5432 (dev)	PostgreSQL with healthcheck
queue-worker	Same as app	none	Processes background jobs
node-dev	node:22-alpine	5173 (dev only)	Vite HMR for live frontend development

4.2 Dockerfile (Multi-Stage)

The Dockerfile uses two stages to produce a lean production image:

Stage 1: node-builder

- Base: `node:22-alpine`
- Copies `package.json` and `package-lock.json` first for layer caching—npm only re-installs when dependencies change.
- Accepts `VITE_APP_NAME`, `VITE_LOCALE`, `VITE_CURRENCY` as build args (Vite inlines these at compile time).
- Runs `npm run build`, outputting hashed assets to `public/build/`.

Stage 2: php

- Base: `php:8.3-fpm-alpine`
- Installs PHP extensions: pdo_pgsql, pgsql, bcmath, mbstring, xml, zip, curl, opcache.
- OPcache tuned for production (`validate_timestamps=0`).
- Composer installs with `--no-dev --optimize-autoloader`.
- Copies built frontend assets from stage 1.
- Optional Xdebug via `--build-arg INSTALL_XDEBUG=true` (excluded by default).
- Runs as `www-data` (non-root) for security.
- Entrypoint caches Laravel config/routes/views and runs migrations.

> **Why not generate APP_KEY in the Dockerfile?**
> The APP_KEY encrypts sessions, cookies, and other sensitive data. Generating it on every container start would invalidate all existing sessions. It must be set once via `.env` and reused across deployments.

4.3 Nginx Configuration

File: `docker/nginx.conf`

Key routing rules:

Path	Handler	Cache
`/build/*`	Static files (Vite)	`Cache-Control: public, immutable` (content-hashed filenames)
`/api/*`	FastCGI → PHP-FPM	None
`/sanctum/csrf-cookie`	FastCGI → PHP-FPM	None (sets XSRF-TOKEN cookie)
`/up`	FastCGI → PHP-FPM	None (health check)

`/*`	`try_files → /index.php`	None (SPA fallback for Vue Router)

Security headers applied globally: `X-Frame-Options`, `X-Content-Type-Options`, `Referrer-Policy`. Dotfiles are denied. Gzip is enabled for text-based assets.

> **No CORS headers needed.**
> Since the Vue SPA and the API are served from the same Nginx on the same domain, all requests are same-origin. Sanctum uses first-party session cookies; no `Access-Control-*` headers are required.

4.4 Docker Compose (Production)

File: `docker-compose.yml`

Key design decisions:

- **Shared `app-public` volume** — Nginx reads static files (`public/build/` , favicon, etc.) directly from this volume instead of proxying them through PHP-FPM. The `:ro` flag ensures Nginx cannot modify files.
- **Database healthcheck** — The `app` service uses `depends_on: db: condition: service_healthy` . This ensures migrations in the entrypoint don't fail because PostgreSQL isn't ready yet.
- **Fail-fast on missing password** — `${DB_PASSWORD:?DB_PASSWORD must be set}` makes Docker Compose refuse to start if the password is not configured, preventing accidental deployment with an empty database password.
- **Queue worker** — Uses the same Docker image but overrides CMD. The entrypoint detects that the command is not `php-fpm` and skips caching/migrations.

4.5 Docker Compose (Development)

File: `docker-compose.override.yml`

Automatically merged when running `docker compose up` (without `-f`).

- **Bind mount** — `.:/var/www/html` maps host code into the container for live editing. An anonymous volume (`/var/www/html/vendor`) preserves the container's Linux-native vendor directory.
- **Entrypoint override** — Skips config/route caching so `.env` changes take effect immediately.
- **Vite HMR** — A `node-dev` service runs the Vite dev server on port 5173. The `@vite` Blade directive auto-detects this via the `public/hot` file.
- **Exposed DB port** — PostgreSQL is accessible at `localhost:5432` for GUI tools.

4.6 Deployment Guide

First-Time Setup

```
# 1. Copy and configure environment
cp .env.example .env
# Edit .env: set DB_HOST=db, DB_PASSWORD=<strong-password>

# 2. Build and start (production)
docker compose -f docker-compose.yml build
docker compose -f docker-compose.yml up -d

# 3. Generate application key
docker compose exec app php artisan key:generate --show
```

```
# Add output to .env as APP_KEY=base64:...
docker compose restart app
```

Development

```
# Start all services (auto-merges override)
docker compose up -d

# App:      http://localhost:8080
# Vite HMR: http://localhost:5173
# Database: localhost:5432
```

Production Deployment

```
# Build with custom env vars
docker compose -f docker-compose.yml build \
  --build-arg VITE_APP_NAME="My Finance App"

# Start (skips dev override)
docker compose -f docker-compose.yml up -d

# View logs
docker compose logs -f app
docker compose logs -f queue-worker
```

Common Operations

```
# Run migrations manually
docker compose exec app php artisan migrate

# Clear all caches
docker compose exec app php artisan optimize:clear

# Access PostgreSQL shell
docker compose exec db psql -U postgres -d finance_app

# Restart queue worker (after code changes)
docker compose restart queue-worker
```

5. File Reference

New Files Created

File	Purpose
`app/Enums/SavingsGoalStatus.php`	PHP enum for savings goal status values
`app/Models/Concerns/BelongsToUser.php`	Global scope trait for user-owned models
`database/migrations/..._add_missing_indexes.php`	Composite indexes for performance
`resources/js/types/router.d.ts`	Vue Router meta type augmentation
`resources/js/utils/api-error.ts`	Typed Axios error handling utilities
`resources/js/utils/constants.ts`	Shared constants (months array)
`Dockerfile`	Multi-stage Docker image (Node + PHP-FPM)
`docker-compose.yml`	Production orchestration
`docker-compose.override.yml`	Development overrides
`.dockerignore`	Excludes unnecessary files from build context
`docker/nginx.conf`	Nginx SPA + API routing config
`docker/entrypoint.sh`	Container boot: caching + migrations

Modified Files

File	Change
`app/Models/Category.php`	Added BelongsToUser trait
`app/Models/Expense.php`	Added BelongsToUser trait
`app/Models/Income.php`	Added BelongsToUser trait
`app/Models/Budget.php`	Added BelongsToUser trait
`app/Models/SavingsGoal.php`	Added BelongsToUser trait + SavingsGoalStatus cast

`app/Http/Controllers/Api/*`	Removed fresh(), removed manual user_id filtering, extracted shared query
`app/Services/DashboardService.php`	Removed userId parameter, uses global scope + shared query
`app/Http/Requests/*/Update*.php`	Added policy-based authorization
`app/Policies/BudgetPolicy.php`	Reformatted to multi-line PSR-12
`app/Policies/SavingsGoalPolicy.php`	Reformatted to multi-line PSR-12
`routes/api.php`	Added rate limiting + scoped nested resource
`bootstrap/app.php`	Added JSON 404 handler for API routes
`resources/js/services/http.ts`	Added error interceptor + timeout
`resources/js/router/guards.ts`	Fixed race condition with shared promise
`resources/js/App.vue`	Added error boundary with fallback UI
`resources/js/types/auth.ts`	Removed unsafe index signature
`resources/js/types/expense.ts`	Added PaymentMethod union type
`resources/js/types/global.d.ts`	Added VITE_LOCALE, VITE_CURRENCY; removed catch-all
`resources/js/utils/formatters.ts`	Reads locale from env vars
`resources/js/components/common/MonthYearPicker.vue`	Added aria-labels, shared MONTHS constant
`resources/js/components/forms/BudgetForm.vue`	Uses shared MONTHS constant
`resources/js/components/forms/ContributionForm.vue`	Deferred form reset via defineExpose
`resources/js/pages/savings/SavingsGoalDetailPage.vue`	Calls resetForm() after successful submission
`resources/js/pages/expenses/ExpenseListPage.vue`	Uses typed error handler
`resources/js/pages/categories/CategoryListPage.vue`	Uses typed error handler
`eslint.config.js`	Changed no-explicit-any from off to warn

`.env.example`	Added VITE_LOCALE, VITE_CURRENCY